(1, 2) Representation and the reasoning algorithms used:

Knowledge is represented using Horn clauses, which are statements containing no more than one positive literal either basic facts like Q and rules like $P_1 \wedge \cdots \wedge P_n \rightarrow Q$. Although both forward and backward chaining resolve entailment in linear time $O(N)$ relative to the size of the knowledge base, I chose backward chaining specifically because it is goal-directed. Forward chaining runs on facts first. It applies modus ponens across every available fact to build out the entire set of possible inferences, that may generate irrelevant proofs before ever touching the user's query. Backward chaining does the opposite. It starts right at the target goal and moves backward, taking only the specific rules and premises required to prove that goal. The search can stop the moment the goal is proven or all valid paths are exhausted, keeping the system from wasting time and compute on massive dense knowledge base.

When a knowledge base moves beyond Horn clauses bringing in things like disjunctions ($\vee$), full negation ($\neg$), exclusive-or constraints ($\oplus$), or multi-variable quantifiers in that case Horn-based backward chaining breaks down. To handle these cases, first-order expressions are translated into Conjunctive Normal Form (CNF) and evaluated using the DPLL (Davis-Putnam-Logemann-Loveland) algorithm. DPLL was picked because it gives us a sound and complete solver that runs faster than brute-force truth tables and resolution theorem proving. Where full resolution tends to explode with an overwhelming number of generated clauses, DPLL keeps the exponential search space manageable by pruning early through unit propagation (resolving deterministic, single-literal clauses right away) and pure literal elimination.

(3) Experiments performed on the public dataset:

Dataset	Samples	Accuracy (%)	Total Time (s)	Avg. Time / Row	Prediction
Ruletaker test dataset	375	279/375 (74.40%)	0.171 s	0.46 ms	False: 322, True: 53
Counterbench train dataset	1200	680/1200 (56.67%)	0.366 s	0.31 ms	False: 1025, True: 175
Folio validation dataset	203	122/203 (60.10%)	44.391 s	218.67 ms	False: 28, True: 50, Uncertain:125

(4) Examples of reasoning traces:

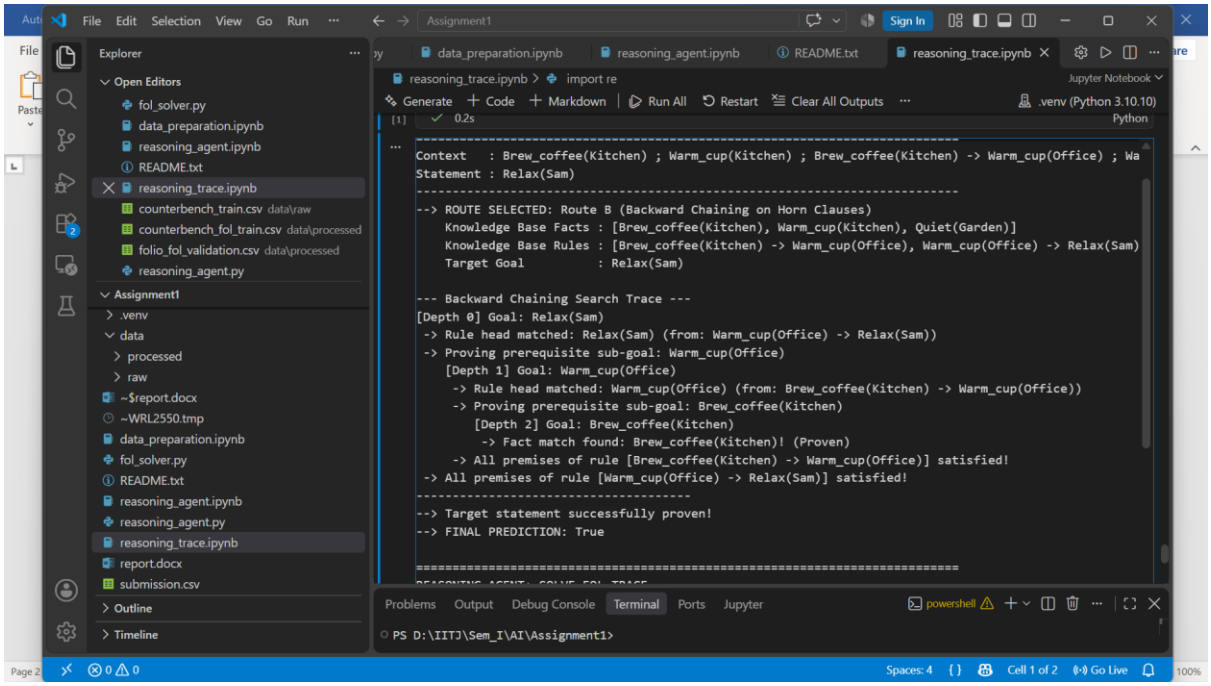
Below output is generated via code (reasoning_trace.ipynb) it self

```
ROUTE SELECTED: Backward Chaining on Horn Clauses
   Knowledge Base Facts: [Brew_coffee(Kitchen), Warm_cup(Kitchen), Quiet(Garden)]
   Knowledge Base Rules: [Brew_coffee(Kitchen) -> Warm_cup(Office), Warm_cup(Office) ->
Relax(Sam)]
   Target Goal: Relax(Sam)

--- Backward Chaining Search Trace ---
[Depth 0] Goal: Relax (Sam)
 -> Rule head matched: Relax (Sam) (from: Warm_cup(Office) -> Relax(Sam))
 -> Proving prerequisite sub-goal: Warm_cup(Office)
   [Depth 1] Goal: Warm_cup(Office)
    -> Rule head matched: Warm_cup(Office) (from: Brew_coffee(Kitchen) -> Warm_cup(Office))
    -> Proving prerequisite sub-goal: Brew_coffee(Kitchen)
```

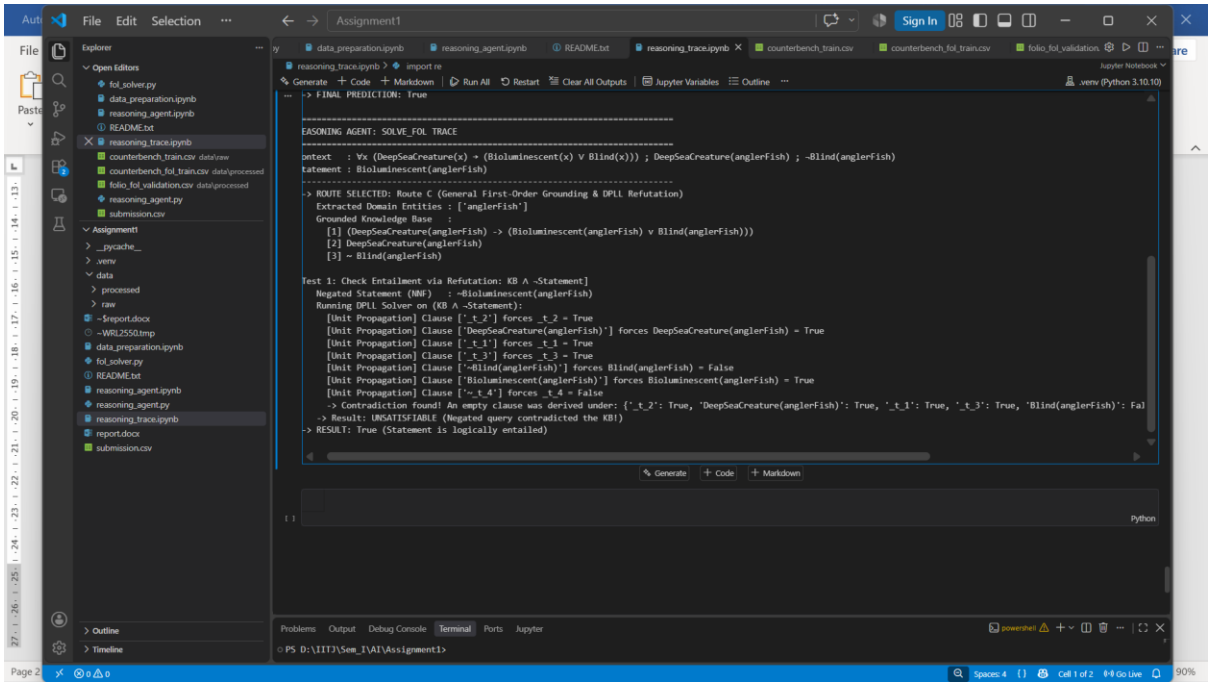
[Depth 2] Goal: Brew_coffee(Kitchen)
-> Fact match found: Brew_coffee(Kitchen)! (Proven)
-> All premises of rule [Brew_coffee(Kitchen) -> Warm_cup(Office)] satisfied!
-> All premises of rule [Warm_cup(Office) -> Relax(Sam)] satisfied!

--> Target statement successfully proven!
--> FINAL PREDICTION: True



--> ROUTE SELECTED: Route C (General First-Order Grounding & DPLL Refutation)
Extracted Domain Entities : ['anglerFish']
Grounded Knowledge Base :
[1] (DeepSeaCreature(anglerFish) -> (Bioluminescent(anglerFish) v Blind(anglerFish)))
[2] DeepSeaCreature(anglerFish)
[3] ~ Blind(anglerFish)

[Test 1: Check Entailment via Refutation: KB ∧ ¬Statement]
Negated Statement (NNF) : ~Bioluminescent(anglerFish)
Running DPLL Solver on (KB ∧ ¬Statement):
[Unit Propagation] Clause ['_t_2'] forces _t_2 = True
[Unit Propagation] Clause ['DeepSeaCreature(anglerFish)'] forces DeepSeaCreature(anglerFish) = True
[Unit Propagation] Clause ['_t_1'] forces _t_1 = True
[Unit Propagation] Clause ['_t_3'] forces _t_3 = True
[Unit Propagation] Clause ['~Blind(anglerFish)'] forces Blind(anglerFish) = False
[Unit Propagation] Clause ['Bioluminescent(anglerFish)'] forces Bioluminescent(anglerFish) = True
[Unit Propagation] Clause ['~_t_4'] forces _t_4 = False
-> Contradiction found! An empty clause was derived under: {'_t_2': True, 'DeepSeaCreature(anglerFish)': True, '_t_1': True, '_t_3': True, 'Blind(anglerFish)': False, 'Bioluminescent(anglerFish)': True, '_t_4': False}
-> Result: UNSATISFIABLE (Negated query contradicted the KB!)
--> RESULT: True (Statement is logically entailed)


File Edit Selection
Assignment1
Sign In
Explorer
Open Editors
fol_solver.py
data_preparation.ipynb
reasoning_agent.ipynb
README.txt
reasoning_trace.ipynb
counterbench_train.csv data\raw
counterbench_fol_train.csv data\processed
folio_fol_validation.csv data\processed
reasoning_agent.py
submission.csv
Assignment1
__pycache__
.venv
data
processed
raw
~$report.docx
~WRL2550.tmp
data_preparation.ipynb
fol_solver.py
README.txt
reasoning_agent.ipynb
reasoning_agent.py
reasoning_trace.ipynb
report.docx
submission.csv
Outline
Timeline
reasoning_trace.ipynb
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
.venv (Python 3.10.10)
-> FINAL PREDICTION: True
EASONING AGENT: SOLVE_FOL TRACE
ontext : ∀x (DeepSeaCreature(x) → (Bioluminescent(x) ∨ Blind(x))) ; DeepSeaCreature(anglerFish) ; ¬Blind(anglerFish)
tatement : Bioluminescent(anglerFish)
-> ROUTE SELECTED: Route C (General First-Order Grounding & DPLL Refutation)
Extracted Domain Entities : ['anglerFish']
Grounded Knowledge Base :
[1] (DeepSeaCreature(anglerFish) -> (Bioluminescent(anglerFish) v Blind(anglerFish)))
[2] DeepSeaCreature(anglerFish)
[3] ~ Blind(anglerFish)
Test 1: Check Entailment via Refutation: KB ∧ ¬Statement]
Negated Statement (NNF) : ¬Bioluminescent(anglerFish)
Running DPLL Solver on (KB ∧ ¬Statement):
[Unit Propagation] Clause ['_t_2'] forces _t_2 = True
[Unit Propagation] Clause ['DeepSeaCreature(anglerFish)'] forces DeepSeaCreature(anglerFish) = True
[Unit Propagation] Clause ['_t_1'] forces _t_1 = True
[Unit Propagation] Clause ['_t_3'] forces _t_3 = True
[Unit Propagation] Clause ['~Blind(anglerFish)'] forces Blind(anglerFish) = False
[Unit Propagation] Clause ['Bioluminescent(anglerFish)'] forces Bioluminescent(anglerFish) = True
[Unit Propagation] Clause ['~_t_4'] forces _t_4 = False
-> Contradiction found! An empty clause was derived under: {'_t_2': True, 'DeepSeaCreature(anglerFish)': True, '_t_1': True, '_t_3': True, 'Blind(anglerFish)': Fal
-> Result: UNSATISFIABLE (Negated query contradicted the KB!)
-> RESULT: True (Statement is logically entailed)
Generate
Code
Markdown
Python
Problems
Output
Debug Console
Terminal
Ports
Jupyter
powershell
PS D:\IIT3\Sem_I\AI\Assignment1>
Spaces: 4
Cell 1 of 2
Go Live